

Plan 002: Remover módulo Redis e dependência morta

Executor instructions: Follow this plan step by step. Run every verification command and confirm the expected result before moving to the next step. If anything in the “STOP conditions” section occurs, stop and report — do not improvise. When done, update the status row for this plan in `plans/README.md`.

Drift check (run first): `git diff --stat 94369a2..HEAD -- src/infrastructure/cache_redis.py src/infrastructure/cache.py requirements.txt src/application/services.py src/ui/streamlit_app.py` Se qualquer arquivo in-scope mudou desde `94369a2`, compare os excertos de “Current state” antes de prosseguir; se houver divergência, PARE.

Status

- **Priority:** P1
- **Effort:** S
- **Risk:** LOW
- **Depends on:** none
- **Category:** tech-debt
- **Planned at:** commit `94369a2`, 2026-06-28

Why this matters

`cache_redis.py` tem 115 linhas com 0% de cobertura de teste. A dependência `redis>=5.0.0` em `requirements.txt` é obrigatória mesmo quando o Redis não está disponível. Na prática, a aplicação sempre cai no fallback (TTLCache em memória). Remover o código morto e a dependência opcional reduz a superfície de manutenção e o `pip install` inicial.

Current state

- `src/infrastructure/cache_redis.py` (115 linhas) — classe `RedisCache` com fallback para `_fallback` (dict), encoder JSON customizado (`_CacheEncoder`) para pandas/numpy/Decimal. Nunca testado (0% cover).
- `src/infrastructure/cache.py` (61 linhas) — classe `TTLCache`, testada (91% cover). Usada como fallback pelo `RedisCache`. Continua existindo.
- `requirements.txt:6` — `redis>=5.0.0`
- `src/ui/streamlit_app.py:12` — `from src.infrastructure.cache_redis import RedisCache`
- `src/ui/streamlit_app.py:37` — `cache = RedisCache(...)` (sempre cai no fallback)

A interface é compatível: tanto `RedisCache` quanto `TTLCache` implementam `get(key)` e `set(key, value)`.

Commands you will need

Purpose	Command	Expected on success
Tests	<code>python -m pytest tests/ -q</code>	all pass
Verificar import	<code>python -c "from src.infrastructure.cache import TTLCache; print('OK')"</code>	OK

Scope

In scope:

- `src/infrastructure/cache_redis.py` — remover (delete file)
- `requirements.txt` — remover `redis>=5.0.0`
- `src/ui/streamlit_app.py` — trocar `RedisCache` por `TTLCache`

Out of scope:

- `src/infrastructure/cache.py` — manter `TTLCache` (é usado pelo `RedisCache` como fallback e continuará sendo o cache ativo)
- `src/application/services.py` — não tocar (recebe cache por injeção, não importa `RedisCache`)
- `src/config/settings.py` — manter `PIFF_CACHE_TTL_SECONDS` (`TTLCache` usa)

Git workflow

- Branch: `advisor/002-remove-redis`
- Commit message: `chore: remove módulo Redis não utilizado e dependência`
- Um commit único (mudança pequena e coesa)

Steps

Step 1: Remover `src/infrastructure/cache_redis.py`

Delete o arquivo:

```
rm src/infrastructure/cache_redis.py
```

Verifique: `ls src/infrastructure/cache_redis.py 2>&1` → No such file

Step 2: Remover `redis>=5.0.0` do `requirements.txt`

Edite `requirements.txt` :

```
- redis>=5.0.0
```

(O `pytest-cov` pode ficar — é útil mesmo que opcional.)

Verifique: `grep -n "^redis" requirements.txt` → sem match

Step 3: Trocar `RedisCache` por `TTLCache` em `streamlit_app.py`

No arquivo `src/ui/streamlit_app.py` :

1. Trocar o import:

```
# Antes:
from src.infrastructure.cache_redis import RedisCache
# Depois:
from src.infrastructure.cache import TTLCache
```

2. Trocar a instanciação em `_build_services()` :

```
# Antes:
cache = RedisCache(ttl_seconds=SETTINGS.cache_ttl_seconds)
# Depois:
cache = TTLCache(ttl_seconds=SETTINGS.cache_ttl_seconds)
```

Verifique: `python -m pytest tests/unit -q` → 12 passed

Step 4: Rodar suite completa

Verifique: `python -m pytest tests/ -q --tb=short` → todos os testes que passavam continuam passando.

Test plan

- Nenhum teste novo — `RedisCache` não tinha cobertura.
- `TTLCache` já tem 91% de cobertura nos testes existentes.
- O `TestReadService` (testado em `test_services.py`) recebe cache por injeção, então a troca de implementação é transparente.

Done criteria

- ☐ [] `src/infrastructure/cache_redis.py` não existe mais
- ☐ [] `redis>=5.0.0` não está em `requirements.txt`
- ☐ [] `grep -rn "RedisCache\|cache_redis" src/` → sem matches
- ☐ [] `python -m pytest tests/unit -q` → 12 passed
- ☐ [] `python -c "from src.infrastructure.cache import TTLCache; print('OK')"` → OK
- ☐ [] Nenhum arquivo fora do escopo foi modificado

STOP conditions

- Se `TTLCache` não tiver um método usado por `RedisCache` que é chamado via duck-typing, PARE e reporte.
- Se algum import indireto de `cache_redis` existir (ex: em `__init__.py`), PARE e reporte o caminho.

Maintenance notes

- Se no futuro o Redis for necessário, a instalação deve ser: `pip install redis` + recriar `RedisCache`. A interface (`get` / `set`) está documentada em `TTLCache` e em `TestReadService`.
- O `TTLCache` atual não faz serialização — só guarda objetos Python em memória. Para Redis seria necessário re-adicionar serialização.